Secure Flutter Apps with Firebase

Authentication, OAuth, and App Check

Part 1: The "Who"

Authentication Fundamentals

AuthN vs. AuthZ

Authentication (AuthN)

Verifying Who You Are.

This is the "gatekeeper" at the front door, responsible for checking a user's credentials. It answers the question: "Is this user really who they say they are?"

Authorization (AuthZ)

Verifying What You Can Do.

This is the "keymaster" inside the building, determining which doors a verified user can open. It answers the question: "Now that I know who this is, what are they allowed to access?"

The Mobile Security Imperative

Sensitive Data

Mobile is the primary interface for financial, health, and personal records. A breach here can be catastrophic.

Personalization & Trust

Personalized experiences require user trust, which is built on the bedrock of secure authentication.

Unique Threat Model

Mobile devices face unique risks from being lost, stolen, or connected to insecure public Wi-Fi networks.

A Taxonomy of Authentication Factors

Factor Type	Description	Common Examples
Knowledge	"Something you know"	Password, PIN, Security Question
Possession	"Something you have"	Phone (for OTP/SMS), USB Key, Auth App
Inherence	"Something unique you are"	Fingerprint, Face ID, Iris Scan

Part 2: The "Service"

Firebase Authentication

The "Build vs. Buy" Dilemma

- **Immense Complexity:** A secure system is more than a password check. It involves secure hashing (bcrypt/scrypt), cryptographic salts, token generation, and rate limiting.
- **Constant Maintenance:** Security is not "set it and forget it." It requires constant vigilance and updates to patch new vulnerabilities.
- **High Cost (Time & Money):** Building and maintaining this system is a full-time job for a dedicated team of specialists.
- **The BaaS Solution (Firebase):** Delegates high-risk components to experts, allowing you to "focus on the rest of your application."

Firebase Authentication Providers

Category	Provider	Flutter Plugin(s) Required
Native	Email & Password	firebase_auth
Native	Phone Number	firebase_auth
Federated (OAuth)	Google	firebase_auth, google_sign_in
Federated (OAuth)	Apple	firebase_auth, sign_in_with_apple
Anonymous	Anonymous Sign-In	firebase_auth

Robust Error Handling

Error Code (e.code)	User-Friendly Message
'user-not-found'	"No user found for that email."
'wrong-password'	"Incorrect password. Please try again."
'email-already-in-use'	"This email address is already in use."
'weak-password'	"Password is too weak. Please use a stronger password."
'invalid-email'	"Please enter a valid email address."

Part 3: Managing the User

Lifecycle

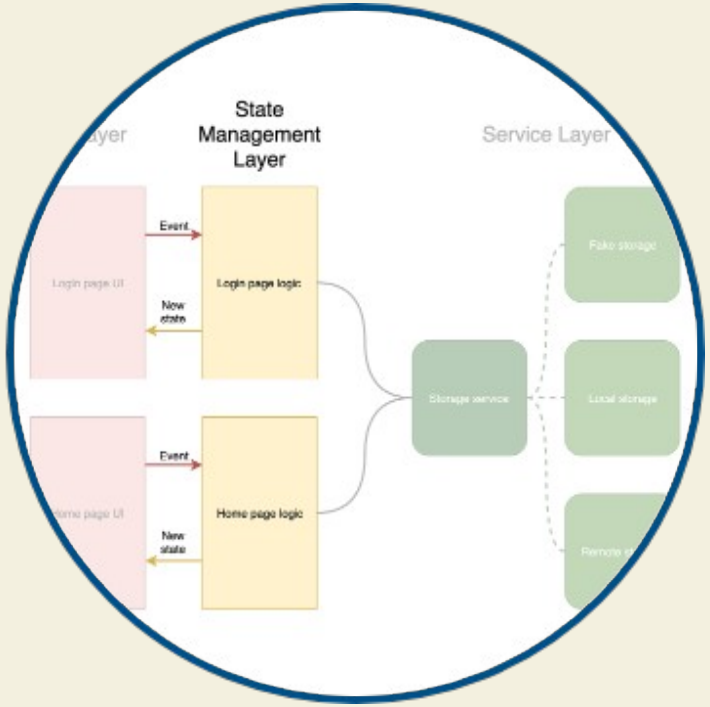
The "Auth Gate" Pattern

Don't Check, Listen

A flawed pattern is checking auth state and pushing a new screen. The robust, declarative solution is to **listen** to the authStateChanges() stream.

This stream emits a new value (User? or null) every time the auth state changes (login, logout, app start).

A StreamBuilder (the "Auth Gate") at the root of your app listens and declaratively builds the correct UI: LoginScreen or HomeScreen.



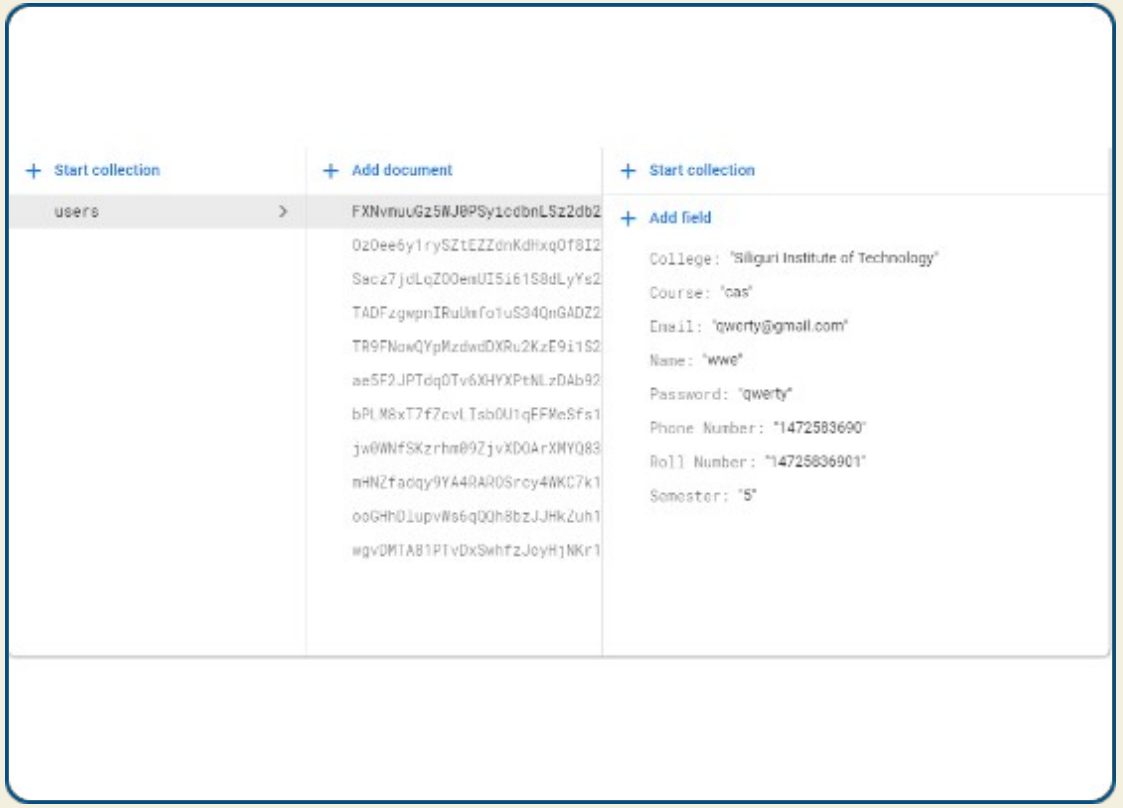
Storing User Profile Data

Decouple Auth from Data

The Firebase User object is for **authentication**, not application data. You cannot add arbitrary fields (like a user bio).

Best Practice:

- Create a users collection in Cloud Firestore.
- After registration, get the uid from the UserCredential.
- Use this uid as the **Document ID** for a new document in your users collection.
- Store all profile data (bio, username, theme) in this document.



Securing Profile Data with Rules

Authorization (AuthZ)

This new data must be secured. We must write Cloud Firestore Security Rules to ensure a user can only read and write **their own** profile document.

The rule uses the request.auth.uid variable (the logged-in user) and compares it to the userId in the document path.

firestore.rules

```
service cloud.firestore {
  match /databases/{database}/documents {
    // Match any document in the "users"
    // collection, {userId} is a wildcard
    match /users/{userId} {

      // Allow read/write ONLY IF the
      // request's auth uid matches the
      // document's ID
      allow read, update, delete:
        if request.auth != null &&
           request.auth.uid == userId;

      // Allow any auth'd user to CREATE
      allow create: if request.auth != null;
    }
  }
}
```

Part 4: The "Easy Button"

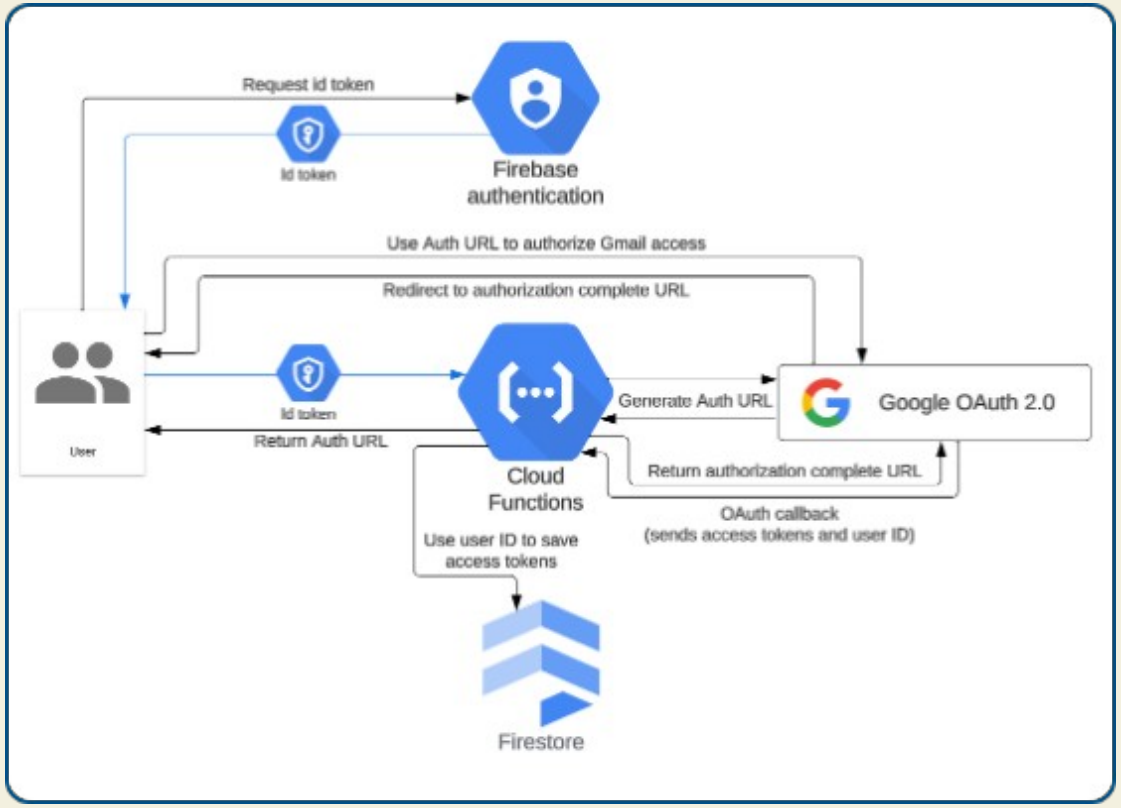
Implementing OAuth Providers

The OAuth "Credential Handoff"

The Two-Step Flow

Firebase trusts the provider (e.g., Google) to have verified the user. The process is a "credential handoff."

1. **App → Provider SDK:** App triggers google_sign_in.
2. **Provider → App:** Google returns an ID Token and Access Token to the app.
3. **App → Firebase:** App creates a GoogleAuthProvider.credential and passes it to Firebase.
4. **Firebase:** Verifies the token and creates its own Firebase session for the user.



Part 5: The "What"

Securing Your Backend with Firebase App Check

The "Other" Security Problem: App Impersonation

The Threat

An attacker decompiles your app, extracts your API keys (e.g., google-services.json), and finds your database URL.

The Attack

They write a script (Python, Node.js) to make requests directly to your backend, completely bypassing your mobile app.

The Impact

Billing Fraud (millions of writes, massive bills) and Data Poisoning (spamming your database with malicious data).

The Three Layers of Firebase Security

Security Layer	Question It Answers	Who It Protects
1. Firebase App Check	"Is this my **real, unmodified app**?"	**The Developer** (from abuse, billing fraud)
2. Firebase Auth	"Is this a **known, valid user**?"	**The User** (from unauthorized access)
3. Security Rules	"Is this user **allowed to do this**?"	**The Data** (from misuse by valid users)

The App Check Development "Catch-22"

- **The Problem:** Production providers (Play Integrity, App Attest) are designed to only validate **real, physical devices**.
- **The Result:** Your Android Emulator and iOS Simulator are **not** real devices. All requests from your dev environment will be blocked.
- **The Solution:** Use the Debug Provider during development.
- **The Workflow:**
  1. Run your app with AndroidProvider.debug.
  2. Look in the debug console for the printed debug token.
  3. Copy/paste this token into the Firebase Console (App Check > Manage debug tokens).